

1. Implémentation du système de stockage centralisé

1.1. Vue d'ensemble de l'architecture

Dans le cadre de ce projet, nous avons choisi d'implémenter trois systèmes de stockage distribué clé-valeur reposant sur des stratégies de partitionnement différentes. La première implémentation, qui fait l'objet de cette section, adopte une architecture centralisée à hachage modulaire.

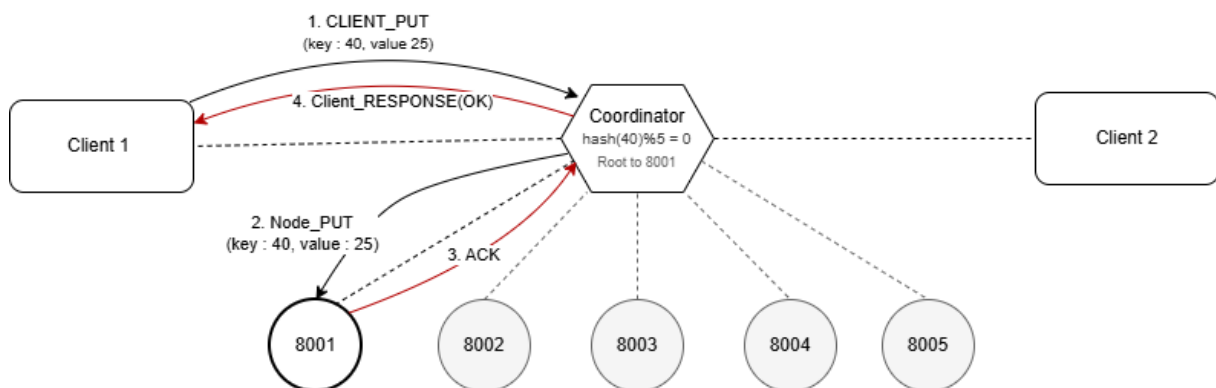
Cette première architecture repose sur deux types de composants :

- Un coordiantor : Un composant unique qui sert de point d'entrée pour toutes les requêtes émises par les clients et applique ces requêtes au nœud dédié
- Des nœuds de stockage : Connu et accessible uniquement par le coordinator, sert à stocker les données des clients.

La distribution des données repose sur une fonction de hachage modulaire : pour une clé donnée, le nœud responsable est identifié par $\text{hash}(\text{key}) \% N$, où N désigne le nombre courant de nœuds actifs.

1.2. Fonctionnement des opérations fondamentales

1.2.1. Opération PUT

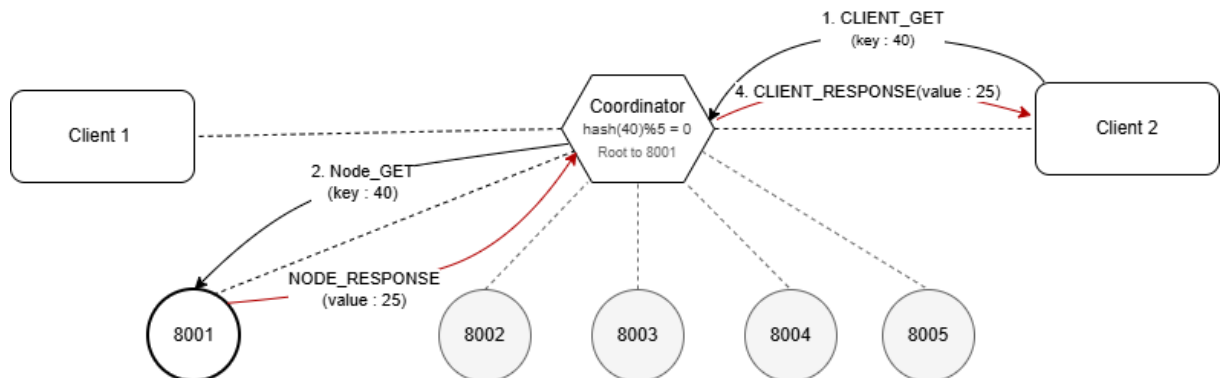


La figure ci-dessus illustre le déroulement d'une requête d'écriture. Lorsque le client 1 souhaite insérer la paire (40, 25), il envoie un message CLIENT_PUT au coordinator. Ce dernier applique la fonction de hachage modulaire $\text{hash}(40) \% 5 = 0$ ce qui désigne le nœud 8001 comme unique responsable de cette clé. Le coordinator transmet alors un message NODE_PUT directement à ce nœud. Le nœud 8001 stocke la paire localement et répond par un ACK. Le coordinator retransmet enfin une CLIENT_RESPONSE(OK) au client pour confirmer le succès de l'opération. On observe que les nœuds 8002 à 8005, représentés en tirets, ne sont à aucun moment sollicités.

À noter que si aucun nœud actif n'est disponible au moment de la requête, le coordinator retourne directement une erreur au client.

Nombre total de message échangé au sein du réseau = 4 (client → coordinator → node → coordinator → client)

1.2.2. Opération GET



La figure ci-dessus illustre le déroulement d'une requête de lecture. Le client 2 envoie un CLIENT_GET portant la clé 40. Le coordinator recalcule $\text{hash}(40) \% 5 = 0$ et identifie de nouveau le nœud 8001 comme responsable. Un NODE_GET est transmis au nœud 8001, qui effectue une recherche dans son store local et retourne la valeur 25 via un NODE_RESPONSE. Le coordinator retransmet enfin une CLIENT_RESPONSE(value : 25) correspondant à la valeur associée à la clé 40.

À noter que si la clé recherchée n'existe pas, le coordinator envoie tout de même un NODE_GET au nœud responsable. Ce dernier effectue la recherche dans son store local, ne trouve pas la clé et retourne un NODE_RESPONSE indiquant que la clé est absente. Le coordinator retransmet alors cette information au client sous la forme d'une CLIENT_RESPONSE signalant que la clé n'existe pas.

Nombre total de Message = 4

1.2.3. Opération Delete

La suppression d'une clé suit exactement la même logique que le GET. Le client envoie un CLIENT_DELETE avec la clé de la donnée que le client souhaite supprimer, le coordinator calcule le nœud responsable et lui transmet un NODE_DELETE. Si la clé existe, le nœud supprime la clé de son store local et indique dans sa réponse au coordinator que la clé a bien été supprimé. Si la clé n'existe pas, le nœud envoie au coordinator que la clé n'existe pas. Le coordinator transmet cette réponse reçue au client avec CLIENT_RESPONSE.

Nombre total de Message = 4

1.3. Gestion dynamique des nœuds

La principale problématique avec un système distribué basé sur un calcul hachage modulaire est la valeur N désignant le nombre de nœuds présent dans le réseau. En effet, si un nœud rejoint ou quitte le réseau, cela modifie le résultat du calcul hachage modulaire et donc provoque un échange massif de flux de donnée. En effet, étant donné que le résultat a

changé les données stockées doivent migrer vers le bon node auxquels ils appartiennent apres la modification du calcule hachage modulaire.

1.3.1. Intégration d'un nouveau nœud (JOIN)

Lorsqu'un nouveau nœud démarre, il s'enregistre auprès du coordinator en envoyant un message REGISTER_NODE. Le coordinator l'ajoute à sa liste de nœuds actifs et déclenche immédiatement un rebalance. En effet, l'ajout d'un nœud fait passer le modulo de N à N+1, ce qui modifie le nœud responsable des clés existantes. Le rebalance ne s'agit pas uniquement d'envoyer les données au nouveau venu. En effet, les emplacements de toutes les données sont recalculés et certaines peuvent migrer entre deux nœuds déjà présents sans impliquer le nouveau nœud.

Le rebalance commence par un inventaire complet : le coordinator envoie un DUMP_REQUEST à chaque nœud actif, qui lui retourne la liste de ses clés. À partir de cet inventaire, le coordinator recalcule pour chaque clé le nœud qui devrait en être responsable avec le nouveau N+1 et construit un plan de migration précis indiquant, pour chaque nœud source, quelles clés doivent être envoyées à quel nœud destination.

Pour chaque entrée de ce plan, le coordinator envoie un message TRANSFER_KEYS au nœud source, en lui communiquant la destination et la liste des clés concernées. Le nœud source collecte les valeurs correspondantes dans son store local, les regroupe en un seul message NODE_BATCH_PUT et l'envoie directement au nœud destination, sans que les valeurs ne transitent par le coordinator. Le nœud destination stocke le batch et répond ACK. Le nœud source supprime alors les clés transférées de son propre store

Pendant toute la durée de ces transferts, les requêtes clients sont temporairement bloquées. Lorsque le plan de migration contient des déplacements à effectuer, le coordinator envoie au préalable un message REBALANCE à tous les nœuds, qui positionnent un flag interne rebalancing = true. Toute requête NODE_PUT, NODE_GET ou NODE_DELETE reçue pendant cet état retourne immédiatement un message REBALANCING au coordinator, qui le propage au client. Ce blocage garantit qu'aucun client ne tente de lire une clé en cours de déplacement, ce qui éviterait une réponse incorrecte de type NOT_FOUND sur une donnée qui existe. À la fin du rebalance, le coordinator diffuse un REBALANCE_DONE à tous les nœuds. A la réception de ce message, les nœuds vont positionner leur flag interne rebalancing = false pour débloquer les requêtes clients.

1.3.2. Retrait d'un nœud (LEAVE)

Lorsqu'un nœud quitte volontairement le réseau, son ShutdownHook envoie un message UNREGISTER_NODE au coordinator. La première action du coordinator, avant de retirer ce nœud de sa liste, est d'envoyer un DUMP_REQUEST pour récupérer l'intégralité des données qu'il détient. Cette étape est critique : si le dump est effectué après la déconnexion, les

données sont définitivement perdues. Une fois les données orphelines récupérées, le coordinator retire le nœud de sa liste et déclenche le rebalance.

Le traitement des données orphelines diffère légèrement du JOIN. Ces données, récupérées par dump avant la déconnexion, sont injectées directement par le coordinator via des messages `NODE_PUT_INTERNAL` vers les nœuds cibles car le nœud source est désormais hors ligne et ne peut plus émettre de `TRANSFER_KEYS`. En revanche, pour les clés déjà présentes sur les nœuds restants qui se retrouvent mal placées suite au changement de modulo de N à $N-1$, le mécanisme est identique au JOIN : le coordinator envoie des `TRANSFER_KEYS` aux nœuds sources, qui transfèrent directement leurs données aux nœuds destinations via `NODE_BATCH_PUT`. Le blocage des clients via le flag `rebalancing` est activé pour les mêmes raisons que lors d'un JOIN.

1.3.3. Détection des pannes par heartbeat

Chaque nœud envoie périodiquement un message `HEARTBEAT` au coordinator toutes les cinq secondes. Un thread de surveillance vérifie régulièrement ces timestamps : si un nœud n'a pas donné signe de vie depuis plus de quinze secondes, il est considéré comme mort et retiré de la liste des nœuds actifs. Le coordinator déclenche alors un rebalance pour redistribuer les clés selon le nouveau N .

A noter que lors du retrait d'un nœud à cause d'une panne, celui-ci quitte le réseau de manière involontaire. Le nœud étant mort, il est impossible d'effectuer un dump préalable de ses données. Les clés qu'il détenait sont donc définitivement perdues. C'est une des limitations de cette implémentation, qui ne dispose pas de mécanisme de réplication.

1.4. Analyse des performances

1.5. Avantages et limites de l'approche centralisée

L'architecture centralisée présente plusieurs avantages notables. Sa simplicité conceptuelle facilite le débogage et la compréhension du système : un seul composant détient la vue globale du cluster, ce qui rend le routage déterministe et prévisible. La cohérence des données est naturellement garantie puisque toutes les décisions de placement transitent par un unique point de calcul. Le rebalance node-to-node améliore par ailleurs la redistribution en évitant que le coordinator ne transporte les données.

Cependant, cette architecture présente des limites structurelles importantes. Le coordinator constitue un point de défaillance unique. En effet, si le coordinator tombe en panne, l'ensemble du système devient inaccessible. L'absence de réplication représente également une faiblesse majeure : en cas de panne d'un nœud, les données qu'il détenait sont irrémédiablement perdues. Enfin, le hachage modulaire classique entraîne une redistribution massive des données à chaque changement de topologie, comportement qui sera

directement comparé à l'approche par hachage cohérent implémentée dans les versions DHT suivantes.